Applied Energy 230 (2018) 1414–1424

Contents lists available at ScienceDirect

Applied Energy

journal homepage: www.elsevier.com/locate/apenergy

Demand response scheduling in industrial asynchronous production lines constrained by available power and production rate

Alemayehu Addisu Desta[a,*], Hakim Badis[b], Laurent George[a,b]

[a] ESIEE, 2 Boulevard Blaise Pascal, 93160 Noisy-le-Grand, France
[b] LIGM (UMR 8049), UPEM, 5 Boulevard Descartes, 77420 Champs-sur-Marne, France

HIGHLIGHTS

- An asynchronous production line modeling based on finite state machine is formulated.
- The problem of maximizing production rates under power constraints is considered.
- Provide a novel framework to manage production lines during demand response intervals.
- A constrained local search heuristic is developed to find near-optimal schedules.
- Simulation results show considerable improvement in production rates.

ARTICLE INFO

Keywords:
Asynchronous production lines
Demand response
Scheduling
Power consumption
Production rate

ABSTRACT

Energy efficiency in factories is a new paradigm arising with Demand Response (DR) programs that enables energy providers to ask their clients to reduce their power consumption for a given time. In this paper, we consider demand side management of an industrial customer who owns asynchronous production line systems. Our aim is to help the customer achieve a good trade-off between production rates and power consumption during DR events. To attain this objective, we develop a framework named *DR-Mgmt* to schedule activities of machines on production lines so that the number of simultaneously working machines is maximized while satisfying DR constraints. Our framework first models activities of a production line using a new approach based on temporal deterministic finite station machine concept, where each state represents machine status (working/idle) and transitions capture temporal changes. Then, the problem of finding an optimal schedule from all feasible schedules is handled by selecting the optimal set of state transitions. We adapt the well-known local search heuristic to find near-optimal transitions. Numerical results show a significant benefit of our approach on production rates during DR intervals. Compared to other approaches, the proposed framework performs best and improves production rates up to 70% in some cases with higher total power consumption. We also validate our work by a real case study.

1. Introduction

Demand Response (DR) allows consumers to play vital roles in the day-to-day operations of electric grids. DR is defined, according to US Federal Energy Regulatory [1], as "changes in electric usage by end-use customers from their normal consumption patterns in response to changes in the price of electricity over time, or to incentive payments designed to induce lower electricity use at times of high wholesale market prices or when system reliability is jeopardized". The DR approach has provided promising solutions in cost savings for different consumer classes such as residential [2], data centers [3], commercial buildings [4], and industries. Since the magnitude of power consumption by manufacturing plants and the change in power that they can provide are generally very large, load flexibility for DR programs could be of a significant amount. For example, a study on assessment of industrial loads for DR shows that top 20 US industries have at least 12 GW of load flexibility [5].

Industrial loads consume large amounts of electricity when compared to other consumers such as residential areas, campus buildings, data centers, etc. According to International Energy Agency, manufacturing plants and residential areas are respectively accounted for 42.0% and 27.1% of global energy consumption in 2015 [6]. In these

* Corresponding author.
E-mail address: alemayehu.desta@esiee.fr (A.A. Desta).

https://doi.org/10.1016/j.apenergy.2018.08.066
Received 7 May 2018; Received in revised form 7 August 2018; Accepted 15 August 2018
Available online 13 September 2018
0306-2619/ © 2018 Elsevier Ltd. All rights reserved.

sectors, a small reduction in energy consumption using Demand Side Management (DSM) could be translated into saving of billions in energy costs per year. Under the umbrella of DSM, DR has gained significant attention in recent years as it demonstrates potentials of reducing energy costs through two programs, namely, Price-based DR (PDR) and Incentive-based DR (IDR). The idea behind PDR is to allow consumers to take advantage of lower price periods by shifting their loads to these times. PDR programs include time of use, real-time pricing, critical load pricing, etc. [7]. IDR programs offer incentives (fixed or time-varying) to encourage consumers reduce their demands. These programs consist of direct load control, interruptible/curtailable services, emergency DR, etc. [7]. In this paper, we consider an IDR scenario where manufacturing plants are rewarded financially according to the amount of their load reductions during DR events.

Under IDR scenario, we consider a manufacturing system where the energy consumers are industrial machines on production lines. Other consumers such as air conditioning and lighting systems are not taken into considerations. We focus on production line systems, specifically Asynchronous Production Lines (ASPL), as these systems are the most common production lines [8]. Our objective is to find schedules of machines on APSL systems to reduce peak power consumption in a DR interval with minimal impact on system production rate if possible. For this purpose, we propose a novel framework named *DR-Mgmt* to manage the relationship between production rate and peak power consumption. After modeling the ASPL's activities by a two dimensional Temporal Deterministic Finite Station Machine (T-DFSM), *DR-Mgmt* looks for extreme schedules: one that maximizes the production rate with no restrictions on power consumption (i.e., default schedule), and the other one that reduces peak power consumption without consideration of production rate (i.e., least feasible schedule). Based on both extreme schedules, *DR-Mgmt* tries to find new schedules enhancing the production rate and ensures that the power consumption is close to the upper limit imposed during DR intervals. The following points summarize the contributions of our work:

- we model activities of an ASPL system using T-DFSM concepts where each state represents machine status (working/idle) and transitions capture temporal changes between states;
- we express schedules as T-DFSM traversals constrained by production rate and power consumption;
- we develop a new framework (*DR-Mgmt*) to manage ASPL functioning with and without power constraints;
- we validate our framework through several numerical results and compare its performance with state-of-the-art solutions.

The remainder of this paper is organized as follows. In Section 2, we provide literature review on related works. Then, Section 3 discusses models of ASPL systems. After this, we explain a scheduling problem and give our solutions in Section 4. We then show in Section 5 numerical results describing the performances of the proposed framework against other approaches. Finally, we conclude this work in Section 6.

2. Related work

DR strategies are mainly based on adjusting operational schedules to reduce demands of electricity. Shoreh et al. [9] provides a comprehensive review on applications of DR in different industry types. Focusing on the IDR, there is a vast number of works in the literature. Zhong et al. [10] proposed a new coupon-based IDR formulation in which load serving entities offer coupon incentives to encourage consumers reduce their demands when there is a potential price spike in the wholesale market. Asadinejad et al. [11] examined a method to design an optimal IDR program which not only minimizes energy costs but also increases the acceptance of the program through finding optimal tariffs at each hour of a day. Vardakas et al. [12] summarized pricing methods and optimization algorithms in both IDR and PDR programs. These works focus on designing DR programs to induce load reduction during peak power consumption periods. However, those programs do not give solutions on how consumers (including industrial facilities) alter their consumption in order to achieve the aforementioned objective which we address in this work.

To help industrial consumers to reduce energy demands in the context of DR, we identify three main approaches from the literature: integrating renewable energy resources, using load aggregators, and rescheduling activities of production lines. The first approach relies on on-site power generations using renewable energy resources (e.g., solar, wind, storage, etc.) to cover some of energy demands so that the customer continues producing while satisfying DR constraints. Wang et al. [13] examined industrial DR scheduling in conjunction with renewable energy resources. Also, Aghaei and Alizadeh [14] provided a literature review on integration of renewable energy resources alongside DR programs. The second approach uses DR aggregators to select some of their customers to relinquish the energy demands during peak periods. Parvania et al. [15] built an optimization framework of DR aggregation that helps DR aggregators offer various contracts for load curtailment and load shifting. The third approach, which is close to our work, explores production scheduling and buffer management of multiple parallel/serial production lines. The main idea is to decide which production line has to work/stop in order to meet DR constraints. The problem to find optimal solutions is known to be very complex [16] and numerous heuristic algorithms have been developed in the literature. Harjunkoski et al. [17] provides a comprehensive review of such techniques. Among these techniques, several research papers propose to use Mixed Integer Linear Programming (MILP) due to its flexibility in handling small [18] and large [19] industrial size problems. There exist other approaches resolving the optimization problem using analytical models and simulations.

For the MILP technique, Fang et al. [20] proposed a general multi-objective MILP for flow shop scheduling problem that considers peak power load and energy consumption. Mansouri et al. [21] also built an MILP model and compared trade-off between minimizing makespan (average task completion time), a measure of service level and total energy consumption. Ramin et al. [22] proposed a framework based on an MILP model considering both production and energy metrics with emphasis on energy electricity markets in case of multiple production lines. As a use case, they applied the developed framework to a real metal casting plant in the Nordic market. For the analytical techniques, Zanoni et al. [23] proposed an analytical model to minimize the costs of producing and storing items while reducing energy consumption in a production line having two machines and three buffers. They analyzed optimal strategies from all possible machine states (Idle, OFF, Working). Zavanella et al. [24] developed analytical model based on queuing theory to find a production schedule with energy constraints including manufacturing plant services. The arrival distribution of the queuing model is related to activation of machines and the service time distribution describes the available power. For simulation models, He et al. [25] proposed a modelling method based on discrete-event simulations to improve energy efficiency in machining manufacturing system by analyzing task-dependant energy consumption. Kohl et al. [26] built a discrete-event simulation model to predict energy consumption of products using load profile of machines in order to reduce energy costs.

The majority of the heuristic algorithms used in those works consider trade-offs between energy consumption and makespan. Moreover, each production line is modelled as a set of virtual machines separated by buffers where each virtual machine is an aggregation of multiple real machines linked with the same conveyor. In contrast to the cited works, our work employees a finer granularity around the real machines. Even though the machines are linked by the same conveyor, we treat each machine separately to find better schedules. In our previous work [27,28], we have considered scheduling on a single conveyor to maximize production rates under a DR constraint. This work generalizes the

Table 1
List of abbreviations.

Abbreviation	Description
ASPL	Asynchronous Production Line
BFS	Breadth-First Search
DFS	Depth-First Search
DR	Demand Response
DR–Mgmt	Demand Response Management
DSM	Demand Side Management
IDR	Incentive-based Demand Response
MILP	Mixed Integer Linear Programming
PDR	Price-based Demand Response
SPL	Synchronous Production Line
T–DFSM	Temporal Deterministic Finite State Machine
VC	Virtual Cell

problem to a case of production lines with multiple conveyors (see Table 1).

3. Asynchronous production line (ASPL) models

In this section, we give a general structure of an ASPL system. Based on this model, we express activities of the ASPL system using a two dimensional T-DFSM. We also present a formal model to characterize the resulting T-DFSM. In the following sections, we use the notations provided in Table 2.

3.1. General case

We start with descriptions of the ASPL system depicted in Fig. 1. The ASPL system is a finite set of Synchronous Production Lines (SPLs) systems interconnected by buffers. Each SPL system consists of multiple machines linked by a single conveyor where machines start processing at the same time but not necessarily finish simultaneously due to differences in their processing times. Consequently, the conveyor of an SPL system is blocked until a machine with maximum processing time finishes working (more details in our previous work [27]). For an ASPL system, we consider the following characterization:

- the ASPL system consists of L SPLs with $L-1$ buffers;
- Each SPL l contains M^l machines linked by a single conveyor;
- the size of ingress Q^0 and outgress Q^L buffers have infinite capacities;
- the size of intermediate buffers (Q^l, $0 < l < L$) can vary from 0 to infinite;
- task τ_i^l is processed on machine M_i^l for duration of $T_{s_i}^l$;
- the machines are **reliable**;
- machine M_i^l consumes p_i^l amount of power when it is processing a task.

Table 2
Summary of main notations.

Symbol	Definition
L	Number of SPLs in the system
VC_i^l	Virtual cell of SPL l at position i
$\text{dist}(VC_i^l, VC_{i+1}^l)$	Distance between VC_i^l and VC_{i+1}^l cell centers in SPL l
K^l	Number of virtual cells in SPL l
M^l	Number of machines in SPL l
M_i^l	Machine i in SPL l
C^l	Capacity of a buffer between SPL l and $l+1$
T_a^l	Inter-arrival time of tasks on SPL l
$T_{s_i}^l$	Processing time of machine i in SPL l
Q^l	An intermediate buffer between SPL l and $l-1$
Q_U^l, Q_D^l	Upstream and downstream buffers of SPL l
a^l, $\overline{a}^l$	Add an item at input of SPL l (a^l) or not ($\overline{a}^l$)
ϵ^l	Status of SPL l (moving or not)
p_i^l	Power consumption of machine i in SPL l when working
$P_{Mi}^l(t)$	Power consumption of machine M_i in SPL l at time t
Δ	Amount of power to reduce in DR interval

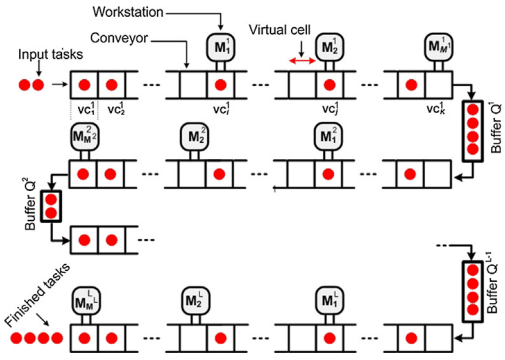


Fig. 1. An asynchronous production line.

3.2. Virtual cell concept

In some real production lines (e.g., food processing lines), pallets are used to move tasks on a conveyor and thus tasks' positions are easy to determine. However, in other production systems (e.g., water bottling production lines), tasks are placed directly on a conveyor without pallets and so positions of tasks are more difficult to obtain. To model task positions and movements independently of conveyor types (with/without pallets), we propose to use a new concept named Virtual Cell (VC). For pallet-based conveyors, each pallet is considered as a VC. Otherwise, we divide a conveyor into multiple VCs which can be seen as a virtual pallet-based conveyor. The distance between two consecutive cells is calculated according to the speed of the conveyor, V_c^l, and task inter-arrival time, T_a^l of SPL l. Let VC_i^l denote a VC at position i, then the distance $dist(\text{VC}_i^l, \text{VC}_{i+1}^l)$ is given by:

$$dist(\text{VC}_i^l, \text{VC}_{i+1}^l) = V_c^l * T_a^l. \tag{1}$$

According to Eq. (1), the time the conveyor needs to travel between two consecutive cells is T_a^l. In order to establish the synchronization (processing at the same time) between machines in the same SPL, their positions must coincide with cell centers. That is, the position of any machine from the 1st VC is a multiple of cell size. More formally, for machine M_Q^l, there exists a strictly positive integer n such that:

$$dist(\text{VC}_1^l, M_Q^l) = n \times dist(\text{VC}_i^l, \text{VC}_{i+1}^l). \tag{2}$$

Since the positions of machines are fixed with respect to the conveyor, the task inter-arrival time must be carefully selected to satisfy Eq. (2). Thus, for $n \in \mathbb{N}^*$ and merging both Eqs. (1) and (2), the value of T_a^l is given by:

$$T_a^l = \frac{dist(\text{VC}_1^l, M_Q^l)}{n \times V_c^l}. \tag{3}$$

3.3. Modelling ASPL using multiple T-DSFMs

A temporal two dimensional T-DFSM is associated to each SPL with global variables which represent conditions on upstream and downstream buffer sizes, Q_U^l and Q_D^l, $l \in L$ and $L \geqslant 1$. The T-DFSM of SPL l is defined as a quintuple $\text{T-DFSM}^l = (\Sigma^l, V^l \times V'^l, (v_0^l, v_{'0}^l), \delta^l, F^l)$ for each SPL, where:

- $\Sigma^l = \Sigma_{V^l} \bigcup \Sigma_{V'^l} = \{a^l, \overline{a^l}\} \bigcup \{\epsilon^l\}$ is the input alphabet,
- V^l is the set of outer states. Each state is coded by a binary value of size K^l bits. Consequently, V^l has a cardinality of $\mathcal{O}(2^{K^l})$,

- V'^l is the set of inner states. Each state is coded by a combination of W (working) and I (idle) symbols with total size M^L. Consequently, V'^l has a of cardinality $\mathcal{O}(2^{M^l})$,
- $(v_0^l, v_0'^l) = (000...0, III...I)$ is the initial state,
- δ^l is the state transition function, δ^l: $(V^l \times V'^l) \times \Sigma^l \rightarrow (V^l \times V'^l)$. It is modeled in a *Precondition:Action| Effect* style where a precondition $\in \{\text{none}^l, t \geqslant c \in \mathbb{R}^+, |Q_D^l| < K^l, |Q_U^l| > 0\}$, Action $\in \{a^l, \overline{a}^l, \epsilon^l\}$, and Effect $\in \{t := c \in \mathbb{R}^+, |Q_U^l|{-}{-}, |Q_D^l| + +\}$. An action can occur if the precondition is satisfied. Consequently, an effect can be executed.
- F is the set of final states. It has only one state, (v_0, v_0').

Our two dimensional T-DFSM^l, $l \in L$, can be represented graphically as one dimensional T-DFSM where each state contains another one dimensional T-DFSM. Indeed, the outer states represent the existence (1) or not (0) of tasks on each VC. Moreover, the inner states represent the status (Working/Idle) of each machine on a conveyor. Hence, we write the outer state with binary values of size K^l, the number of cells, and the inner states by a word of size M^l, number of machines. According to these concepts, transitions can be intra- and inter-outer states. Intra-state transitions express the temporal behaviour of machines on a conveyor from working (W) to idle (I) during task processing. When all machines finish processing, they will go to idle state and the value can be expressed as $(I)^{M_l}$. The inter transitions between outer states represent movement of conveyors with actions such as insert (a^l) or not ($\overline{a}^l$). If a finished task is present at the last VC and a downstream buffer is full, $|Q_D^l| = K^l$, we stay in the same outer and inner states because in this case moving the conveyor will add a new task to the downstream buffer. In the same manner, if upstream buffer is empty, $|Q_U^l| = 0$, the outer and inner states do not change, i.e., a loop exists. Moreover, inter-outer state transitions can happen without a precondition. In this case, we express this situation by none^l.

For example, let us consider an ASPL system with two SPLs ($L = 2$) as shown in Fig. 2. Each SPL contains two machines, $M^1 = 2$ and $M^2 = 2$ that have their own processing times T_{s1}^1, T_{s2}^1, T_{s1}^2 and T_{s2}^2, respectively. The machines are placed on consecutive VCs of their respective conveyors. Each SPL has two buffers: upstream and downstream. The upstream buffer of SPL 1 and downstream buffer of SPL 2 are assumed to have unlimited capacity. The shared buffer is a downstream buffer for SPL 1 and an upstream for SPL 2. This buffer has a total capacity of C^1. The effective capacity ($|Q^1|$) can be updated for each arriving task from SPL 1 if the buffer is not full (i.e., if $|Q^1| \leqslant C^1$: $|Q^1| + +$) and retrieving a task by SPL 2 if the buffer is not empty (i.e., if $|Q^1| > 0$: $|Q^1|{-}{-}$). At the head of conveyor l, three actions are possible: insert (a^l), not insert ($\overline{a}^l$) and do nothing (ϵ^l). The first two actions are associated with movement of conveyors and so they are based on the conditions of buffers. The last action does not require conveyors' movements. We can describe the activities of SPL 1 and 2 by T-DFSM^1 (see Fig. 3) and T-DFSM^2 (see Fig. 4), respectively. Both T-DFSMs have the same states because the two SPLs are similar (the same number of VCs and machines). As described before, each global state is represented by outer and inner states. The outer states {00, 01, 10, 11} indicate the presence or absences of tasks in the two VCs. Inside each outer state, sub T-DFSM is defined using inner states. Each sub T-DFSM describes the temporal behaviour of machines. For instance, when we enter the outer state (11) in both T-DFSMs, all the machines are working (WW) simultaneously. If they have the same processing time, $T_{s1}^1 = T_{s2}^1$ or $T_{s1}^2 = T_{s1}^2$, they will finish at the same time and go to inner state (II). Otherwise, the machine with the smallest processing time go to idle and the other one continues working. Hence, we go from state (WW) to (WI) or (IW).

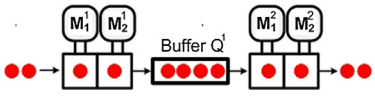


Fig. 2. An example of asynchronous production line.

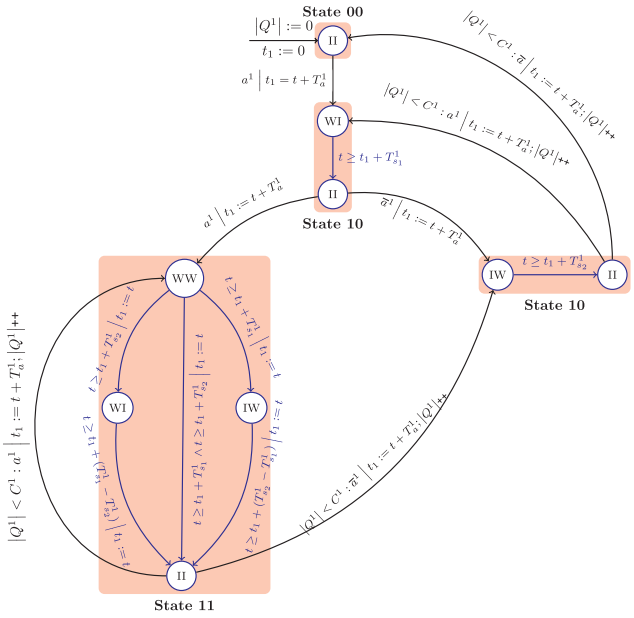


Fig. 3. A T-DFSM associated to the first SPL.

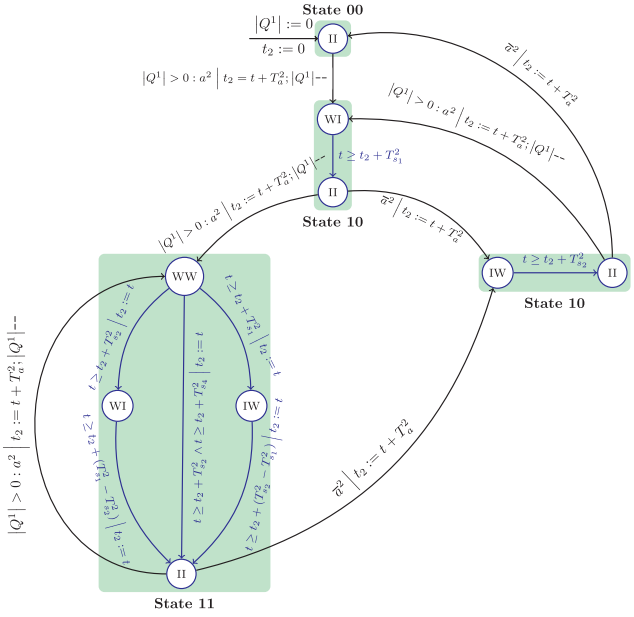


Fig. 4. A T-DFSM associated to the second SPL.

When the machine with longest processing time finishes, we go to state (II). Moreover, we save current times on each transition. When T-DFSM^1 or T-DFSM^2 is in the inner state (II), we can do an inter-outer state transition. This happens if the shared buffer conditions are satisfied by moving the conveyor and doing actions such as insert or not insert a new task at the input of the SPLs.

3.4. Modelling ASPL using combined T-DFSMs

All T-DFSMs, T-DFSM^l, $l \in L$, can be combined into a single two dimensional T-DFSM, denoted by T-DFSM^{all} = T-DFSM^1 ‖ T-DFSM^2 ‖ ⋯ ‖ T-DFSM^L. The T-DFSM^{all} is defined as quintuple T-DFSM^{all} = $(\Sigma, V \times V', (v_0, v_0'), \delta, F)$ for each SPL, where:

- $\Sigma = \Sigma^1 \bigcup \Sigma^2 \bigcup \cdots \bigcup \Sigma^L = \{a^l, \overline{a}^l, \epsilon^l | l = 1, 2, \cdots L\}$ is the set of input alphabets,
- $V = V^1 \circ V^2 \circ \cdots \circ V^L = \{v^1 v^2 \cdots v^L | v^1 \in V^1 \wedge v^2 \in V^2 \cdots \wedge v^L \in V^L\}$ is an

ordered concatenation of outer states. The size of each state is $\sum_{l=1}^{L} K^l$. Consequently, the cardinality of V can be given by $\mathcal{O}(2^{\sum_{l=1}^{L} K^l})$,

- $V' = V'^1 \circ V'^2 \circ \cdots \circ V'^L = \{v'^1 v'^2 \cdots v'^L | v'^1 \in V'^1 \wedge v'^2 \in V'^2 \cdots \wedge v'^L \in V'^L\}$ is an ordered concatenation of inner states. The size of each state is $\sum_{l=1}^{L} M^l$. Consequently, the cardinality of V' can be given by $\mathcal{O}(2^{\sum_{l=1}^{L} M^l})$,
- $(v_0, v'_0) = (v_0^1 v_0^2 \cdots v_0^L, v_0'^1 v_0'^2 \cdots v_0'^L)$ is the initial state,
- δ is the state transition function, $\delta: (V \times V') \times \Sigma \rightarrow (V \times V')$. It is modeled in a Precondition-Action-Effect style where a precondition follows: $t \geqslant c \in \mathbb{R}^+ \vee \bigwedge_{l=1}^{L} (\text{none}^l \vee |Q_D^l| < C^l \vee |Q_U^l| > 0 \vee (|Q_D^l| < C^l \wedge |Q_U^l| > 0))$, Action can be represented as $\bigwedge_{l=1}^{L} (\epsilon^l \vee a^l \vee \overline{a}^l)$, and Effect is a combination of elements in $\{t^l := c^l \in \mathbb{R}^+, |Q_U^l|--, |Q_D^l|++\}$ set, $l = 1, 2, ...L$;
- F is the set of final states. It has only one state, (v_0, v'_0).

As in single T-DFSM^l (see Section 3.3), T-DFSM^{all} maintains inner and outer state concepts. Outer/inner states are results of concatenation operation of the different outer/inner states of single T-DFSM^l. Inter-outer state transitions happen when one or more events/actions are produced under specific conditions. Consequently, we can have two types of inter-outer state transitions: synchronous and asynchronous. Synchronous inter-outer states transitions occur when two or more SPLs perform actions simultaneously. It means, at least 2 or more a^l and/or $\overline{a}^l$ actions appear in the transition syntax $\left(\bigwedge_{l=1}^{L} (\epsilon^l \vee a^l \vee \overline{a}^l)\right)$. Otherwise, inter-outer state transitions are considered as asynchronous. Obviously, an action should be realized when the precondition is verified regarding downstream and upstream buffers. Without constraints, a precondition on SPL l is defined as none^l. We formulate the combination of all inter-outer state transition conditions as $\bigwedge_{l=1}^{L} (\text{none}^l \vee |Q_D^l| < C^l \vee |Q_U^l| > 0 \vee (|Q_D^l| < C^l \wedge |Q_U^l| > 0))$. The effect of this operation can: (a) update upstream and downstream buffers, $|Q_U^l|--$ and/or $|Q_D^l|++$, (b) conserve the processing times of the still working machines, (c) move the conveyor for a time unit of T_a^l for SPL l. In intra-outer (or inter-inner) state, transitions depend only on processing times of different machines. This is why only the time is taken into consideration, $t \geqslant c \in \mathbb{R}^+$. By analogy to single T-DFSM^l case, no action is needed for intra-outer state transitions, none^l. As output, we save the current time to determine the remaining time of working machines.

Fig. 5 shows a small part of the combination of T-DFSM^1 (Fig. 3) and T-DFSM^2 (Fig. 4). The initial state is given by outer state (0000) and inner state (*IIII*). From this state, we can either do synchronous or asynchronous transitions to outer states (1000) and (1010), respectively. The synchronous transition appears when adding a task only on the first SPL without conditions on downstream buffer which leads to the following ≪ *Precondition*: *Action*≫ style on inter-outer state transitions: none$^1 \wedge$ none^2: $a^1 \wedge \epsilon^2$. On the other hand, the asynchronous transition realized by adding task simultaneously on both SPLs, $a^1 \wedge a^2$. Obviously, the shared buffer needs to have at least one task, $|Q^1| > 0$. Arriving to the outer state (1000), we enter to the inner state (*WIII*) which indicates that the first machine of the SPL 1 is working and all the others are in idle state. From this state two events are possible: inserting a task to SPL 2 while the first machine is still working (allows to go to another outer state) or finish by going to inner state (*IIII*) (no change of outer state) after processing time $T_{s_1}^1$. Every time we do an outer transition, we move the conveyor and conserve the remaining processing times of still working machines. To go from the inner state (*WIII*) of outer state (1000) to inner state (*WIWI*) of outer state (1010), we need to insert a task (a^2) on SPL 2. This is possible only if the shared buffer is not empty, $|Q^1| > 0$. As a result, $|Q^1|--$.

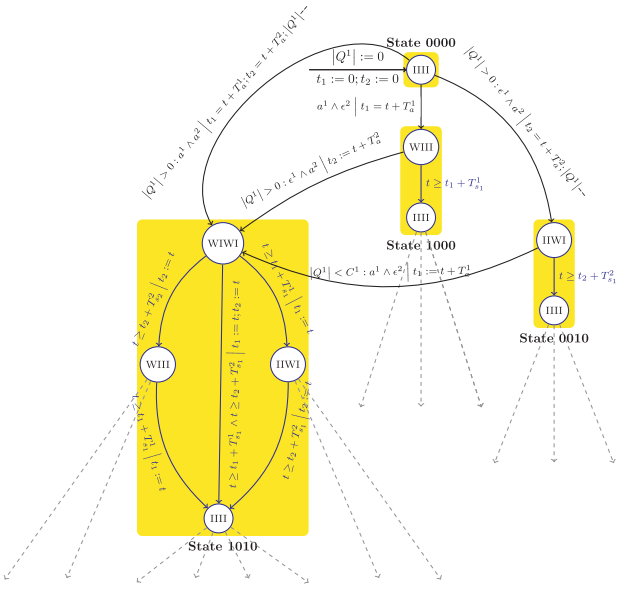


Fig. 5. A T-DFSM associated to the combined SPLs.

4. Scheduling problem

In this section, we focus on power consumption reduction subject to DR constraints. We propose a framework based on the local search heuristic algorithm to find schedules. Furthermore, we analyze its complexity.

4.1. Main concept

Our aim is to reduce the total power consumption of an ASPL system according to the available power and to keep higher production rates as much as possible. We know that tasks are inserted at the head of each conveyor and a machine works only if there is a task under it. If we make decisions to stop task insertions, the conveyor will be purged progressively and machines will go to idle state waiting for other tasks to arrive. In this case, the power consumption of the system will progressively decrease and will approach to 0 kW which is good in terms of minimizing power consumption but impossible to continue producing tasks. This decision is appropriate when we want to stop producing (at the end of the day for example). On the other hand, intensive insertion of tasks will increase the production rate and consequently increase the total power consumption of the system. This decision is good under normal conditions when we do not take care of DR. Hence, the two previous decisions represent extreme situations: stop inserting or inserting as much as possible. Our objective is to adapt insertion of tasks in all SPLs by dynamically varying input insertion rates in such a manner that at every instant the total power consumption of simultaneously working machines is less than the available power. This process should also ensure the least impact on the production rate.

4.2. T-DFSM^{all} traversal and complexity analysis

A *schedule* in an ASPL system is a finite sequence of decisions. Each decision is formed by a conjunction of actions performed at conveyors' inputs: $\bigwedge_{l=1}^{L} (\epsilon^l \vee a^l \vee \overline{a}^l)$. We recall that an action on SPL l, $0 < l \leqslant L$, consists of inserting a new task (denoted by symbol a^l), not inserting ($\overline{a}^l$) and do nothing (ϵ^l). Each decision in a schedule generates a move between two states in T-DFSM^{all}. More production rate means more task insertions in each decision and so traversed states will contain more number of 1s. It is obvious that at least one outer state from the subset

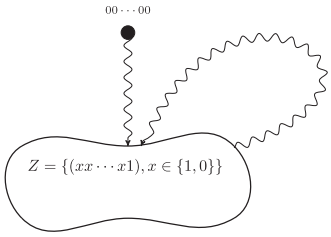


Fig. 6. T-DFSM^{all} traversal.

$\mathscr{Z} = \{(xx\cdots x1), x \in \{1, 0\}\}$, is necessarily visited at least once during traversals. Each state from Z allows to discharge a finished task out of the system. Consequently, the problem of finding a schedule maximizing production rates can be seen as a T-DFSM^{all} traversal problem to reach an outer state from Z with the maximum number of $x = 1$ and then try again continuously. Fig. 6 illustrates such a process.

The time complexity to find the optimal path from a given state to a state in Z depends on the number of inter-outer state transitions except for those belonging to $\mathscr{Z}$ ($V-\mathscr{Z}$) and the number of transitions. For computing the time complexity, we consider the subset of outer states Z as a single outer state. Consequently, the total number of outer states in a T-DFSM^{all} is $2^{\sum_{l=1}^{L} K^l - 1} + 1$. As our T-DFSM^{all} is implemented as an adjacency matrix, if we use an exhaustive search algorithm based on, for example, DFS [29] or BFS [30], then the time complexity can be given by $\mathcal{O}((\text{number of outer states})^2)$, so it becomes $\mathcal{O}(2^{2\sum_{l=1}^{L} K^l})$. Since this complexity is exponential in the number of outer states, we rely on a well-known local search heuristic algorithm [31] to find in a bounded time a simple global solution (not necessarily optimal) based on local optimum.

4.3. Scheduling under normal conditions

Under normal conditions (no constraints on power), the only objective is to maximize production rates. We apply the local search heuristic algorithm on our T-DFSM^{all} traversal. From an outer state, we move to a designated successor outer state, and we repeat this process for each visited state. A successor outer state is selected as designated among all candidate successor states if, under satisfying precondition, the decision on its corresponding transition contains the highest task insertions. It is obvious that the best decision is $a^1 \wedge a^2 \wedge \cdots \wedge a^L$. To break ties, we select the transition having tasks insertion from upstream buffers with the highest fullness percentage defined as $\frac{bufferlength}{buffercapacity} \times 100$. The underlying reasons are to reduce the queuing time of tasks and to avoid blocking the SPL just before the full buffer.

To illustrate the working of principle of the heuristic algorithm, we use the ASPL shown in Fig. 2. We suppose that $T^1_{s_1} = T^1_{s_2} = 20$ s, $T^2_{s_1} = T^2_{s_2} = 2$ s, and $T^1_a = T^2_a = 2$ s. We have two possible cases: (a) when shared buffer has null capacity, and (b) when the shared buffer has non-null capacity. The first case is well studied in our previous work [28]. In the second case, SPLs are asynchronous and serially correlated with each other by shared buffers. Fig. 7a shows T-DFSM^{all} (of the example depicted in Fig. 2) with only outer states and different line colors representing synchronous and/or asynchronous transitions. The resulting default schedule of our local search heuristic algorithm is illustrated by Fig. 7b. As shown in the figure, we move from outer state (0000) until state (1110) with similar fashion of the above case. At this last state, due to shorter processing times of SPL 2, tasks are processed and moved quickly on SPL 2 while SPL 1 is still processing tasks. Consequently, the SPL 2 goes to empty most of the time and stay in state (1100) waiting for new tasks. When a task is available in the shared buffer (upstream buffer of SPL 2), we return to the state (1110) and so on.

4.4. Scheduling under DR

The default schedule takes transitions without considering limits on total power consumption of involved machines. Since the entry inner state of each outer state has the maximum symbols W (working), temporal peaks of power consumption occur when an ASPL system enters into outer states. We define the peak power consumption of an ASPL system during a given schedule as the highest peak power among all traversed outer states. More formally, let p_i^l denote the power consumption value of machine M_i^l, $0 < l \leqslant L$ and $0 < i \leqslant M^l$, v and v'_{in} denote an outer state and its entry inner state, respectively. We associate to each entry inner state a vector map h replacing each symbol W by 1 and symbol I by 0. The peak power consumption of an ASPL when it is on an outer state v is given by:

$$p(v) = \sum_{l=1}^{L} \sum_{i=1}^{M^l} p_m^l h_i^l. \tag{4}$$

From the example of Fig. 2, if we consider that all machines have equal power consumption of 100 kW, the peak power consumption on each outer state is shown in Fig. 7b. The initial outer state (0000) has the least peak power consumption (0 kW) and the state (1111) has the highest peak power consumption (400 kW). The peak power consumption of an ASPL depends on the used schedule. It is given by the highest peak power consumption among all visited outer states selected by the schedule. We can see from Figs. 7b that the related peak power consumption is 300 kW. Let V_{seq} be the subset of outer states visited by a schedule *seq*. The peak power consumption of an ASPL using a schedule *seq* can be expressed as:

$$p(seq) = \max_{v \in V_{seq}} p(v) \tag{5}$$

Assume that a DR signal is emitted demanding to reduce power consumption by an amount Δ for a specified window duration. In this interval, the default schedule may not be feasible due to its peak power consumption which can exceed a DR threshold. For example, the default schedule in Fig. 7b is not feasible if the new available power in DR interval is 200 kW. In this case, we need to find a new schedule having the peak power consumption in the DR interval under the new available power threshold. For this purpose, we propose a new framework, named *DR-Mgmt*, which is composed of 4 steps. The objective of the first two steps is to verify the feasibility of extreme schedules in the DR interval. The third step is to find a schedule better than the least feasible schedule. The last step is to determine the sequence of transitions from the current default schedule to the new schedule obtained in step 3. The steps are detailed in the following paragraphs.

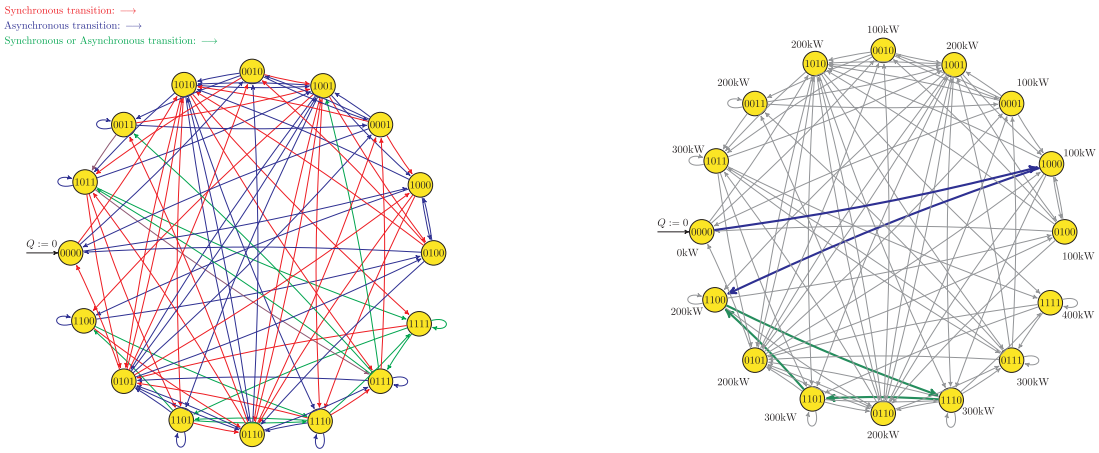
Step 1: If the peak power consumption of default schedule does not exceed the new available power in DR interval, then this schedule is the best solution. Hence, the DR does not impact the production. Otherwise, go to step 2 below.

Step 2: We verify that if the least feasible schedule satisfy the new available power in DR interval based on the following lemma.

Lemma 1. *A feasible schedule exists if the power consumption of every machine is less than the available power.*

Proof. See our paper [27]. □

If the constraints imposed by Lemma 1 are verified ($p_i^l \leqslant$ *available power*), a least feasible schedule consists of only one machine working at a time in the system. This can be realized by inserting a task on the first SPL and waiting until it leaves the last SPL before the next insertion. The T-DFSM^{all} traversal consists of going from (000···0), (100···0), (010···0) until (000···1) and then going back to (100···0) and so. The least feasible has the least production rate since we have one task at a time in the system. If the lemma's constraints are not



(a) T-DFSM^{all} for both SPLs

(b) Default schedule in T-DFSM^{all}

Fig. 7. Scheduling based on local search heuristic algorithm when shared buffer $C^1 = 2$.

verified, no feasible schedule exists and we are forced to stop the system.

Step 3: If the least feasible schedule exists, this step tries to enhance the production rate by selecting other possible feasible schedules that can consume just less than the DR threshold. For this purpose, we extend the local search heuristic algorithm to take into account power consumption of successors. So, the designated successor state will not only be selected based on the number of insertions (see Section 4.3) but also the peak power consumption on it should respect the DR constraint. When no successor state can be selected as designated due to DR constraint, the local search algorithm is restarted by backtracking to the previously traversed states.

Step 4: This step helps the ASPL system to make a transition from normal condition mode to a DR mode. In other words, it determines how to go from the default schedule used under normal conditions to the new feasible schedule obtained by step 3 under DR. We propose to apply the well-known Dijkstra algorithm [32] on T-DFSM^{all} to find the shortest path in terms of number of transitions between the current state and one of the subset of states used by the feasible schedule. The shortest path ensures small transition times to DR.

Fig. 8 shows the *DR-Mgmt* running on the ASPL of Fig. 2 configured with $T^1_{s_1} = T^1_{s_2} = 20$ s, $T^2_{s_1} = T^2_{s_2} = 2$ s, $T^1_a = T^2_a = 2$ s, and $C^1 = 2$. The default schedule used in normal conditions is depicted in Fig. 8a with thick lines. Its peak power consumption is 300 kW. We suppose that in DR interval, $\Delta = 100$ kW. So, the available power is 200 kW. Applying *step* 1, we see that the power consumption of the system when the default schedule used exceeds the newly available power in DR. In *step* 2, the obtained feasible schedule helps the system to consume only 100 kW (see Fig. 8b). Then, *step* 3 tries to improve the least feasible schedule by using local search heuristic algorithm. The obtained schedule is illustrated in Fig. 8c. Lastly, *step* 4 computes the shortest path from the current state of the ASPL system under normal condition (for example, say (1110)) to one of the subset states traversed by the new feasible schedule {(1000), (1100), (0110), (0101), (0100), (1010), (0010), (1001)}. Fig. 8d shows that the shortest path is from (1110) to (0101) with only one hop.

5. Numerical results and discussions

In this section, we investigate numerically the functioning of our DR-Mgmt framework and its performance. We consider two cases: a simple example of ASPL and a real-world application.

5.1. DR-Mgmt framework functioning

In order to describe the schedules under normal and DR conditions, we use the exemplary SPL configuration shown in Fig. 2. It has 4 machines on 4 VCs, a shared buffer with capacity C^1 and $T^1_a = T^2_a = 4$ s. All time units are given in seconds. We assume that each machine consumes $p^l_i = 100$ kW ($i = 1, 2$ and $l = 1, 2$) when it processes a task.

5.1.1. Under normal conditions

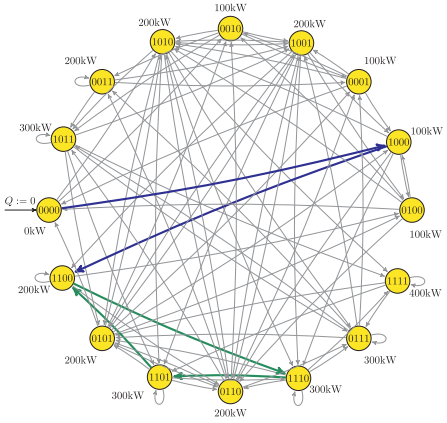
In this case, we do not impose constraints on power consumption. Here, the main constraint is the availability of intermediate buffer between SPL 1 and SPL 2. In the case where the buffer size is null, the system becomes a single SPL and the movement of conveyors are synchronized with the machine which has the maximum processing time. This scenario is well investigated in our previous work [28]. Let us now assume that the buffer size is non-null and fixed to $C^1 = 2$. Table 3 illustrates default schedules found by our framework with different processing times. When $T^1_{s_i} = T^2_{s_i}$, both SPLs are synchronized and can insert simultaneously tasks. When $T^1_{s_i} < T^2_{s_i}$, SPL 1 can continue inserting tasks while SPL 2 is working. Thus, we have different actions on both SPLs, i.e., actions can be $a^1 \wedge a^2$, $a^1 \wedge \epsilon^2$, $\epsilon^1 \wedge a^2$ and $\epsilon^1 \wedge \epsilon^2$ (gray colored rows).

5.1.2. Under DR constraints

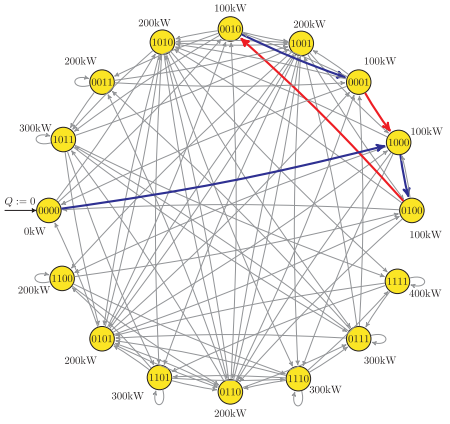
The ASPL system is currently using a default schedule and takes the loop on state (1111). Its total power consumption is 400 kW. Under DR interval, this system cannot support reducing its power consumption by an amount Δ higher than 300 kW (feasibility condition) since each machine consumes 100 kW. When $\Delta = 300$ kW, there exists a least feasible schedule because this power is enough for only one single working machine at time in the system. When $\Delta = 200$ kW, we can find a better schedule than the least feasible one according to *step 3* of the framework. Table 4 shows the resulting schedule that satisfies the DR constraint of $\Delta = 200$ kW. The gray colored rows of the table show state transitions in DR window and loops in the new schedule. Also, we can see that the schedule moves from state (1111) of the default schedule to the nearest state (0101) of the new schedule using the shortest path computed by *step 4*.

5.2. Performance comparisons with other approaches

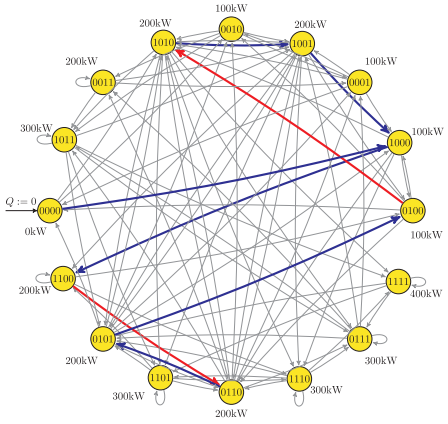
To examine the efficiency of our approach, we compare its performances (production rate and power consumption) with two other



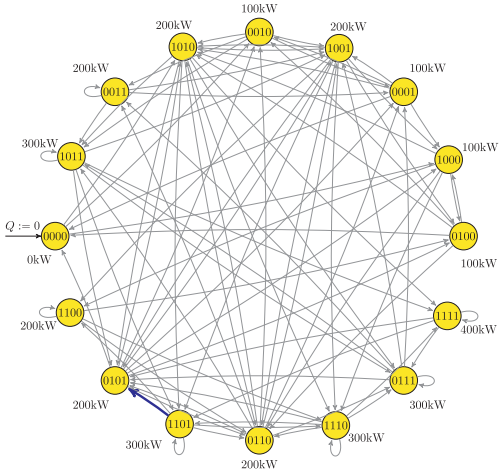
(a) Step 1: default schedule



(b) Step 2: least feasible schedule



(c) Step 3: Better schedule



(d) Step 4: Using Dijkstra

Fig. 8. T-DFSM and scheduling when buffer size is $C^1 = 2$.

approaches developed in the literature. The first approach named *Strgy 1* [21] aims at reducing makespan without considering production rate metric. It aggregates the machines of each SPL to one virtual machine and considers a case of only single task in the system at a time. Unlike *Strgy 1*, the second approach named *Strgy 2* [20] tries to enhance production rate under the power consumption constraints by inserting multiple consecutive tasks to one virtual machine till the shared buffer is full. We have conducted four sets of simulations based on the ASPL system shown in Fig. 2. In each set, we fix the ASPL parameters such as processing times, power consumption and buffer size and vary the amount of power to save (Δ) in DR (from 0 to 400 kW). Comparative simulation results are shown in Table 5. From the table, we can see that when Δ is 0 kW (i.e., no DR), all the strategies achieve the same performances on production rate and power consumption metrics. In this case, the system provides the best production rates and consumes the highest power. When we increase Δ, our strategy has the best performance in terms of production rates while meeting the DR constraints. The improvement in production rate of our approach reaches 70% and 60% in some cases (e.g., Δ = 100 kW) when compared to *Strgy 1* and *Strgy 2*, respectively. We note, however, that the total power consumption using our strategy is always higher than that of the two strategies. This can be explained by the fact that our strategy enhances the production rate by going close to the limit of the available power. We also remark that when Δ is increased, the production rates of all

Table 3
Actions and machine status when buffer size is $C^1 = 2$.

$4 = T^1_{s_i} = T^2_{s_i}$			$8 = T^1_{s_i} > T^2_{s_i} = 4$			$4 = T^1_{s_i} < T^2_{s_i} = 8$		
Outer state	Action	Inner State	Outer state	Action	Inner state	Outer state	Action	Inner state
0000	$\epsilon^1\epsilon^2$	IIII	0000	$\epsilon^1\epsilon^2$	IIII	0000	$\epsilon^1\epsilon^2$	IIII
1000	$a^1\epsilon^2$	WIII	1000	$a^1\epsilon^2$	WIII	1000	$a^1\epsilon^2$	WIII
1000	$\epsilon^1\epsilon^2$	IIII	1000	$\epsilon^1\epsilon^2$	IIII	1000	$\epsilon^1\epsilon^2$	IIII
1100	$a^1\epsilon^2$	WWII	1100	$a^1\epsilon^2$	WWII	1100	$a^1\epsilon^2$	WWII
1100	$\epsilon^1\epsilon^2$	IIII	1100	$\epsilon^1\epsilon^2$	IIII	1100	$\epsilon^1\epsilon^2$	IIII
1110	a^1a^2	WWWI	1110	a^1a^2	WWWI	1110	a^1a^2	WWWI
1110	$\epsilon^1\epsilon^2$	IIII	1110	$\epsilon^1\epsilon^2$	WWII	1110	$\epsilon^1\epsilon^2$	IIWI
1111	a^1a^2	WWWW	1110	$\epsilon^1\epsilon^2$	IIII	1110	$a^1\epsilon^2$	WWII
1111	$\epsilon^1\epsilon^2$	IIII	1111	a^1a^2	WWWW	1111	ϵ^1a^2	IIWW
1111	a^1a^2	WWWW	1111	$\epsilon^1\epsilon^2$	WWII	1111	$a^1\epsilon^2$	WWWW
1111	$\epsilon^1\epsilon^2$	IIII	1111	$\epsilon^1\epsilon^2$	IIII	1111	$\epsilon^1\epsilon^2$	IIII

Table 4
State transitions when Δ = 200 kW and buffer size $C^1 = 2$.

$4 = T^1_{s_i} = T^2_{s_i}$			$8 = T^1_{s_i} > T^2_{s_i} = 4$			$4 = T^1_{s_i} < T^2_{s_i} = 8$		
Outer state	Inner state	Power(kW)	Outer state	Inner state	Power(kW)	Outer state	Inner state	Power(kW)
1111	WWWW	400	1111	WWWW	400	1111	WWWW	400
1111	IIII	0	1111	WWII	200	1111	IIWW	200
0101	IWIW	200	1111	IIII	0	1111	IIII	0
0101	IIII	0	0101	IWIW	200	0101	IWIW	200
1010	WIWI	200	0101	IWII	100	0101	IIIW	100
1010	IIII	0	0101	IIII	0	0101	IIII	0
0101	IWIW	200	1010	WIWI	200	1010	WIWI	200

Table 5
Performance comparisons of different strategies where buffer size $C^1 = 2$, Rate in tasks/min, Power in kW.

SPL 1				SPL 2				Δ	DR-Mgmt		Strgy 1		Strgy 2	
$T^1_{s_1}$	$T^1_{s_2}$	p^1_1	p^1_2	$T^2_{s_1}$	$T^2_{s_2}$	p^2_1	p^2_2		Rate	Power	Rate	Power	Rate	Power
4	4	100	100	8	8	100	100	0	5	400	5	400	5	400
4	4	100	100	8	8	100	100	100	5	300	1.5	100	2	200
4	4	100	100	8	8	100	100	200	2.5	200	1.5	100	2	200
4	4	100	100	8	8	100	100	300	1.5	100	1.5	100	0	0
4	4	100	100	8	8	100	100	400	0	0	0	0	0	0
8	8	100	100	4	4	100	100	0	5	400	5	400	5	400
8	8	100	100	4	4	100	100	100	3.75	300	1.5	100	2	200
8	8	100	100	4	4	100	100	200	3.75	200	1.5	100	2	200
8	8	100	100	4	4	100	100	300	1.5	100	1.5	100	0	0
8	8	100	100	4	4	100	100	400	0	0	0	0	0	0
4	4	100	100	4	4	100	100	0	7.5	400	7.5	400	7.5	400
4	4	100	100	4	4	100	100	100	5.625	300	1.875	100	2.5	200
4	4	100	100	4	4	100	100	200	3.75	200	1.875	100	2.5	200
4	4	100	100	4	4	100	100	300	1.875	100	1.875	100	0	0
4	4	100	100	4	4	100	100	400	0	0	0	0	0	0
8	4	100	50	4	8	50	100	0	5	300	5	300	5	300
8	4	100	50	4	8	50	100	100	5	200	1.5	100	1.77	150
8	4	100	50	4	8	50	100	200	1.5	100	1.5	100	0	0
8	4	100	50	4	8	50	100	300	0	0	0	0	0	0

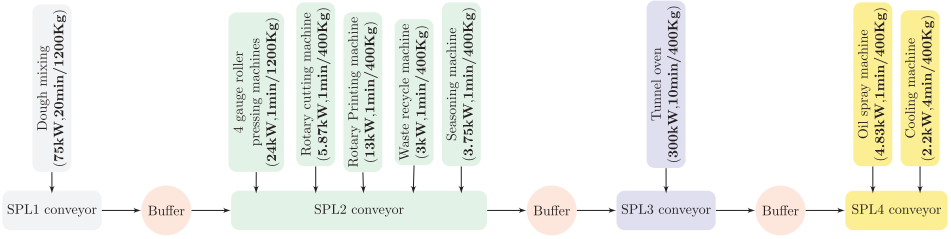


Fig. 9. Soft/hard biscuits production line of HG-SHB.

strategies are decreased.

5.3. Applicability and performance analysis of DR-Mgmt in real case application

Let us assume that a company XYZ produces soft/hard biscuits using the production line manufactured by HG-SHB company [33]. For example, this production line is one of production lines used by Lefèvre Utile (LU) [34] based in Nantes, France. The production line consists of four SPLs separated by three buffers. The process of producing biscuits consists of dough mixing, pressing and forming, flavoring, baking, oil spraying and cooling processes. Each process is running on a dedicated machine and has its own parameters: power consumption and processing time. Fig. 9 shows the production line and its technical parameters taken from the datasheet.

Under normal circumstances (i.e., without power constraints), the target production rate is 400 kg per 20 min which is equivalent to 1200 kg/h. This production rate is specified in the datasheet as the maximum allowed. Table 6 illustrates the time required to complete each 400 kg batch size of hard biscuits. We can see that cycles of 20 min occur after the first batch.

Table 6
HG-SHB soft/hard biscuit production line functioning under normal conditions.

Batch No.	Arrival time (min)				
	SPL1	SPL2	SPL3	SPL4	Out
1	0	20	22	32	37
2	20	40	42	52	57
3	40	60	62	72	77
4	60	80	82	92	97
⋮	⋮	⋮	⋮	⋮	⋮

We assume that the company XYZ subscribes to an IDR program offered by an energy provider. During few hours of a day, the energy provider can ask the company to reduce power consumption by offering incentives. The objective of our *DR-Mgmt* framework is to help the company respect the DR constraints with least possible impact on

Table 7
State transitions in HG-SHB production line with and without DR.

Without DR			With DR of Δ = 75kW		
Outer State	Inner state	Power consumption (kW)	Outer State	Inner state	Power consumption (kW)
000000000	IIIIIIII	0	000000000	IIIIIIII	0
100000000	WIIIIIII	75	100000000	WIIIIIII	75
110000000	WWIIIIII	99	110000000	WWIIIIII	99
111000000	WWWIIIII	104.87	111000000	WWWIIIII	104.87
111100000	WWWWIIII	117.87	111100000	WWWWIIII	117.87
111110000	WWWWWIII	120.87	111110000	WWWWWIII	120.87
111111000	WWWWWWII	124.62	111111000	WWWWWWII	124.62
101111000	WIWWWWII	100.62	101111000	WIWWWWII	100.62
100111000	WIIWWWII	94.75	100111000	WIIWWWII	94.75
100011000	WIIIWWII	81.75	100011000	WIIIWWII	81.75
100001000	WIIIIWII	78.75	100001000	WIIIIWII	78.75
100000100	WIIIIIWI	375	100000000	WIIIIIII	75
100000010	WIIIIIIW	79.83	000000100	IIIIIIWI	300
100000001	WIIIIIIW	77.2	000000010	IIIIIIWI	4.83
100000000	WIIIIIII	75	000000001	IIIIIIIW	2.2
			100000000	WIIIIIII	75

production rates. Let us now consider implementing our framework in the company and investigate its performance in comparison with other approaches from the literature. In order to manage the HG-SHB soft/hard biscuit production line, our *DR-Mgmt* framework requires a T-DFSM^{all} model that represents the temporal activities and states of machines on the production line. For this purpose, we define 9 VCs, each containing one machine. 2^9 outer states are generated to capture the presence (1) or absence (0) of tasks on these VCs. We consider each batch of 400 kg as one task. Also, each outer state includes 2^9 inner states representing the state (Working/Idle) of every machine.

Under normal conditions (without DR), the production line functioning is expressed by a default schedule between outer states (110000000) and (100000000) as shown on the left hand side of Table 7. It should be noted that we cannot reach state (111111111) (i.e., all machines working at the same time) due to differences in processing times of dough mixer and other machines. In this configuration, the maximum power consumption is 375 kW which can be attained when dough mixer and tunnel oven are working simultaneously. The production rate is determined by the dough mixer which has the highest processing time. Hence, the production rate is found to be one batch (400 kg) per 20 min or 1200 kg per hour that coincides with the indicated value on the datasheet.

Under DR constraints and feasibility conditions, our *DR-Mgmt* framework proposes alternative schedules to support reducing power consumption. Indeed, since the least power consumption to guarantee production is 300 kW (feasibility condition) and the maximum power consumption is 375 kW, the company can support reducing its power consumption by an amount of Δ ($0 \leqslant \Delta \leqslant 75$ kW) to remain above the 300 kW limit. Otherwise, the production should be interrupted. On the right hand side of Table 7, an alternative schedule (found by our framework) for $\Delta = 75$ kW is shown. We can see that the production process is not interrupted and the new maximum power consumption is 300 kW.

There are different approaches that can be applied by the company XYZ to deal with DR constraints. As detailed in Section 5.2, *Strgy 1* and *Strgy 2* approaches are the most relevant for the context of our work. In that section, based on a general set of parameters and scenarios, we have shown that our *DR-Mgmt* framework achieves better production rates under different DR constraints when compared to *Strgy 1* and *Strgy 2* approaches. Now, we need to confirm these results in case of real production line. Table 8 illustrates performances of the company in terms of power consumption, production rate and monetary gains. Under normal conditions ($\Delta = 0$) or out of feasibility conditions ($\Delta > 75$ kW), all approaches achieve the same performances. Otherwise ($0 \leqslant \Delta \leqslant 75$ kW), our *DR-Mgmt* framework has the best performance which is up to 31% improvements in production rate when compared to *Strgy 1* and *Strgy 2* approaches. For profit comparison, we define two metrics: incentive gain and overall gain. The incentive gain can be defined as payments that an energy provider refunds a subscribed consumer for reducing energy consumption during DR intervals. If we assume that the company XYZ receives an incentive of 20% for not produced biscuits and 1Kg of biscuits costs $35, its incentive gain can by expressed as (production under normal conditions - production under DR interval) × 20% × unit price. It is obvious that our *DR-Mgmt* framework achieves the least benefits, up to 31% less on incentive compared to the other approaches, due to its higher production rate. Unlike incentive gain, the overall gain accumulates incentive and effective production gains. It is formulated as (production under DR × unit price) + incentive gain. As *DR-Mgmt* framework maximizes the effective production rates under DR constraints, it achieves high overall gain, up to 22%, compared to the other approaches.

To integrate our framework in the production line of the company XYZ, we need an automation control system such as Supervisory Control and Data Acquisition (SCADA) [35] which is widely used in industries to monitor real-time sensor data and remotely control industrial processes. Otherwise, it is very complex to handle schedules manually due to low error tolerance and time requirements in schedule execution. The DR–Mgmt framework can be implemented and deployed

Table 8
Performance comparisons of different strategies.

Δ (kW)	DR-Mgmt				Strgy 1				Strgy 2			
	Rate (kg/h)	Power (kW)	Incentive gain ($)	Total gain ($)	Rate (kg/h)	Power (kW)	Incentive gain ($)	Total gain ($)	Rate (kg/h)	Power (kW)	Incentive gain ($)	Total gain ($)
0	1200	375	0	42,000	1200	375	0	4200	1200	375	0	42,000
50	705.9	300	3458.7	28165.2	486.5	300	4994.5	22,022	600	300	4200	25,200
75	679.25	300	3645.25	27,419	486.5	300	4994.5	22,022	600	300	4200	25,200
100	0	0	8400	8400	0	0	8400	8400	0	0	8400	8400

in the SCADA server. It continuously learns about the current state of production line such as machine status, buffer sizes, positions of tasks, etc., by receiving real-time data from sensors. When a DR signal is received, the DR–Mgmt computes a new schedule and the sequence of decisions are then sent to devices responsible for inserting tasks on conveyors' inputs.

6. Conclusions

This work addressed the challenge of reducing power consumption of industrial customers with minimal impact on production rates during Demand Response (DR) intervals. A framework that achieves a good trade-off between production rates and power consumption in context of industrial production lines is developed. In contrast of related works, this framework includes two related parts: modeling of production lines and finding schedules. For the first part, we proposed a model based on a Temporal Deterministic Finite State Machine (T-DFSM) to capture activities of machines on production lines. The second part dealt with T-DFSM traversal problem to find feasible schedules. To this end, we adapted the heuristic local search algorithm to take into account constraints on power consumption while maximizing production rates. We also explored the computation complexity of the modified heuristic algorithm. Finally, we compared the performance of the proposed framework to other approaches from the related work in case of general scenarios and real-world application. Considering production rate metric, it is shown that our framework performs best and achieves up to 70% and 31% more than the other approaches in the two cases, respectively.

For future work, we would like to enhance the proposed scheduling algorithm. This could involve developing a new algorithm based on linear programming to find near optimal schedules and compare it with the proposed algorithm. Furthermore, considering case of unreliable machines is also an interesting extension of our work. Other manufacturing types such as assembly production lines could be investigated in the future work. Finally, we plan to deploy and test our approach in real industrial facilities by integrating the framework inside a SCADA system.

References

[1] Lee M, Aslam O, Foster B, Kathan D, Kwok J, Medearis L, et al. Assessment of demand response and advanced metering. Federal Energy Regulatory Commission, Tech. Rep.

[2] Muratori M, Rizzoni G. Residential demand response: dynamic energy management and time-varying electricity pricing. IEEE Trans Power Syst 2016;31(2):1108–17.

[3] Liu Z, Liu I, Low S, Wierman A. Pricing data center demand response. ACM SIGMETRICS 2014;42(1):111–23.

[4] Hao H, Corbin CD, Kalsi K, Pratt RG. Transactive control of commercial buildings for demand response. IEEE Trans Power Syst 2017;32(1):774–83.

[5] Starke M, Alkadi N, Ma O. Assessment of industrial load for demand response across us regions of the western interconnect. Tech. rep. Oak Ridge, TN (United States): Oak Ridge National Lab. (ORNL); 2013.

[6] Birol F. Key world energy statistics. < https://www.iea.org/publications/freepublications/publication/KeyWorld2017.pdf > .

[7] Albadi MH, El-Saadany EF. A summary of demand response in electricity markets. Electr Power Syst Res 2008;78(11):1989–96.

[8] Li J, Blumenfeld DE, Huang N, Alden JM. Throughput analysis of production systems: recent advances and future topics. Int J Prod Res 2009;47(14):3823–51.

[9] Shoreh MH, Siano P, Shafie-khah M, Loia V, Catalão JP. A survey of industrial applications of demand response. Electr Power Syst Res 2016;141:31–49.

[10] Zhong H, Xie L, Xia Q. Coupon incentive-based demand response: theory and case study. IEEE Trans Power Syst 2013;28(2):1266–76.

[11] Asadinejad A, Tomsovic K, Varzaneh MG. Examination of incentive based demand response in western connection reduced model. North American power symposium (NAPS), 2015. IEE; 2015. p. 1–6.

[12] Vardakas JS, Zorba N, Verikoukis CV. A survey on demand response programs in smart grids: pricing methods and optimization algorithms. IEEE Commun Surv Tutorials 2015;17(1):152–78.

[13] Wang X, El-Farra NH, Palazoglu A. Optimal scheduling of demand responsive industrial production with hybrid renewable energy systems. Renew Energy 2017;100:53–64.

[14] Aghaei J, Alizadeh M-I. Demand response in smart electricity grids equipped with renewable energy sources: a review. Renew Sustain Energy Rev 2013;18:64–72.

[15] Parvania M, Fotuhi-Firuzabad M, Shahidehpour M. Optimal demand response aggregation in wholesale electricity markets. IEEE Trans Smart Grid 2013;4(4):1957–65.

[16] Cataldo A, Perizzato A, Scattolini R. Production scheduling of parallel machines with model predictive control. Cont Eng Prac 2015;42:28–40.

[17] Harjunkoski I, et al. Scope for industrial applications of production scheduling models and solution methods. Comput Chem Eng 2014;62:161–93.

[18] Merkert L, Harjunkoski I, Isaksson A, Säynevirta S, Saarela A, Sand G. Scheduling and energy–industrial challenges and opportunities. Comput Chem Eng 2015;72:183–98.

[19] Velez S, Merchan AF, Maravelias CT. On the solution of large-scale mixed integer programming scheduling models. Chem Eng Sci 2015;136:139–57.

[20] Fang K, Uhan N, Zhao F, Sutherland JW. A new approach to scheduling in manufacturing for power consumption and carbon footprint reduction. J Manuf Syst 2011;30(4):234–40.

[21] Mansouri SA, Aktas E, Besikci U. Green scheduling of a two-machine flowshop: trade-off between makespan and energy consumption. Eur J Oper Res 2016;248(3):772–88.

[22] Ramin D, Spinelli S, Brusaferri A. Demand-side management via optimal production scheduling in power-intensive industries: the case of metal casting process. Appl Energy 2018;225:622–36.

[23] Zanoni S, Bettoni L, Glock CH. Energy implications in a two-stage production system with controllable production rates. IJPE 2014;149:164–71.

[24] Zavanella L, Zanoni S, Ferretti I, Mazzoldi L. Energy demand in production systems: a queuing theory perspective. IJPE 2015;170:393–400.

[25] He Y, Liu B, Zhang X, Gao H, Liu X. A modeling method of task-oriented energy consumption for machining manufacturing system. J Clean Prod 2012;23(1):167–74.

[26] Kohl J, Spreng S, Franke J. Discrete event simulation of individual energy consumption for product-varieties. Proc CIRP 2014;17:517–22.

[27] Desta A, Badis H, George L, Courbin P. An efficient production scheduling based on queuing theory in systems with synchronous part transfer during a demand response event. In: SmartGridComm; 2017. p. 546–52.

[28] Desta A, Badis H, George L. Demand response scheduling in production lines constrained by available power. In: IEEE ICC ICT4SG; 2018.

[29] Tarjan R. Depth-first search and linear graph algorithms. SIAM J Comput 1972;1(2):146–60.

[30] Zhou R, Hansen EA. Breadth-first heuristic search. Artif Intell 2006;170(4–5):385–408.

[31] Savelsbergh MW. An efficient implementation of local search algorithms for constrained routing problems. EJOR 1990;47(1):75–85.

[32] Dijkstra EW. A note on two problems in connexion with graphs. Numer Math 1959;1(1):269–71.

[33] HG-SHB. Shb1200 soft/hard biscuits production line; 2018.

[34] LU, Lefvre-utile (lu) biscuit. < https://www.lu.fr/ > .

[35] Erez N, Wool A. Control variable classification, modeling and anomaly detection in Modbus/TCP SCADA systems. IJCIP 2015;10:59–70.